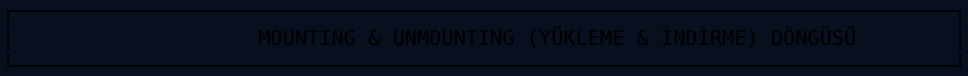
44

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Carrier Thread, Mounting & Unmounting: Loom Non-Blocking I/O ile Nasıl Park Eder?

Carrier Threads, Mounting & Unmounting: Epoll Non-Blocking I/O Integration

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic




```
Virtual Thread (VT-1) ──► Carrier Thread üzerinde çalışıyor (MOUNTED)
   │
   ▼ socket.read() çağrıldı! (Veri henüz ağdan gelmedi)
JVM Continuation.yield() Tetiklenir:
   1. VT-1'in stack frame'leri HEAP'e kopyalanır (UNMOUNTED).
   2. Soket Epoll / Kqueue olay dinleyicisine kaydedilir.
   3. Carrier Thread anında BOŞA ÇIKAR ve VT-2'yi çalıştırmaya başlar! (0 CPU Kaybı!)
   │
   ▼ Ağdan veri geldi! (Epoll Interrupt)
JVM Zamanlayıcı:
   1. VT-1'in stack frame'leri boşta olan herhangi bir Carrier Thread'e geri yüklenir.
   2. VT-1 kaldığı yerden sanki hiç durmamış gibi devam eder! (MOUNTED Tekrar!)
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Satranç şampiyonunun aynı anda 50 kişiyle körleme maç yapmasını düşünün. Şampiyon (Carrier Thread) senin masana gelir, hamleni yapar (Mounting). Sen 'Düşüneyim…' dediğinde (I/O bloklaması), şampiyon masanın başında 10 dakika dikilip seni beklemez! Not defterine pozisyonu yazar ve hemen diğer masaya koşar (Unmounting). Sen hamleni yaptığında elini kaldırırsın, şampiyon boş kaldığı an masana geri dönüp devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a Grandmaster playing simultaneous chess against 50 opponents. The Grandmaster (Carrier Thread) steps up to your board and evaluates your move (Mounting). If you say 'Let me think for 2 minutes' (Blocking I/O), the Grandmaster does not stand idly waiting! He jots down your board coordinates in a pocket notebook and sprints to the next table (Unmounting). When you finally make your move, you raise your hand; the Grandmaster resumes your match on his next pass!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Continuation ve Non-Blocking Hook İç Mekaniği: 1) Continuation.yield(): Sanal thread bloklayan bir I/O işlemine (ör. `SocketChannel.read()`, `Thread.sleep()`) girdiğinde, HotSpot JDK soketin bloke olduğunu anlar. `VirtualThread.park()` çağrılır ve düşük seviyeli `Continuation.yield()` çalışır. 2) Yığın Ayrıştırma (Stack Freezing): O anki Java çağrı yığını Carrier thread'in C++ yığınından sökülüp JVM Heap'e taşınır. Carrier thread işletim sistemi açısından hiçbir zaman bloklanmaz; ForkJoinPool'daki diğer hazır sanal thread'leri çalıştırmaya devam eder. 3) Epoll/Kqueue Entegrasyonu: JVM'nin dahili Poller thread'i Linux `epoll` veya Mac `kqueue` üzerinden soketin hazır olduğunu duyunca sanal thread'i `RUNNABLE` yapar. 4) Yeniden Yükleme (Thawing): Boş bir carrier thread yığını heap'ten geri yükleyip (thawing) bytecode yürütmesine devam eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Continuation Mechanics and Epoll Integration: 1) `Continuation.yield()`: When a virtual thread encounters blocking I/O (e.g., `Socket.getInputStream().read()`, `LockSupport.park()`), the Java runtime intercepts the call. Instead of performing a kernel syscall that blocks the OS thread, it invokes `VirtualThread.park()`. 2) Stack Freezing: The virtual thread's call frames are copied from the carrier thread's native stack onto the JVM heap. The carrier thread is immediately disassociated (unmounted) and returns to its `ForkJoinPool` work-stealing loop. 3) Poller & Reactor Registration: The file descriptor is registered with HotSpot's global I/O Poller thread (using Linux `epoll` or macOS `kqueue`). 4) Thawing & Resumption: Upon I/O readiness, the Poller re-enqueues the virtual thread. An available carrier thread thaws the stack frames from the heap and resumes instruction execution transparently.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Carrier Thread Geçişini Gösteren Canlı Test
import java.util.concurrent.Executors;

public class CarrierThreadHandoffDemo {
    void main() throws InterruptedException {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            executor.submit(() -> {
                // I/O öncesi Carrier Thread kimliğini yazdır
                System.out.println("1. Bloklama Öncesi: " + Thread.currentThread());

                try {
                    // Bu çağrı unmount eder; Carrier Thread boşa çıkar!
                    Thread.sleep(200);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

                // I/O sonrası: Farklı bir Carrier Thread tarafından mount edilmiş olabilir!
                System.out.println("2. Bloklama Sonrası: " + Thread.currentThread());
            });
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Sanal thread'ler CPU-bound (yoğun matematik, video işleme, büyük veri sıkıştırma) işler için HİÇBİR performans artışı sağlamaz! Aksine unmount/mount overhead'i yüzünden platform thread'lerinden daha yavaş kalırlar. Sanal thread'lerin tek süper gücü I/O-bound (Ağ, DB, Disk, REST API) bloklamalarını sıfır maliyete indirmektir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Virtual threads provide ZERO throughput gains for CPU-bound computational workloads (e.g. video rendering, cryptography, dense matrix math). Context freezing and thawing actually introduces measurable overhead. Virtual threads are designed exclusively to eliminate concurrency ceilings caused by blocking I/O (databases, HTTP microservices, message queues).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sanal thread bloklandığında (I/O) Continuation.yield() ile unmount edilir, Carrier thread boşa çıkar. Epoll uyanınca yeniden mount edilir.

🎯 Key Takeaways for the Interview

• When a virtual thread blocks on I/O, `Continuation.yield()` unmounts its stack to the heap, freeing the carrier thread. Epoll notifications re-mount it seamlessly.